

ETX: Making Fusion Win

Author: Kailun Chen · Status: v1 plan · Date: 2026-09-25 · Evidence: examples/llm/results/bench_2026-09-25.log, docs/ETX_Technical_Design.md sections 15.4-15.6

0. Summary

- **Goal 1 (must):** with the same tiles, the ETX megakernel runs at least 15% faster than one kernel per operator replayed from a HIP graph. Today it is 6-15% slower.
- **Goal 2:** on the three Qwen models, at least 15% faster than vLLM’s best configuration. Today it is about 2x slower.
- **Goal 3:** every token identical to Hugging Face greedy, deterministic across runs.
- **Route:** first make the barriers cheap (device-wide barriers per layer from 8 down to 2), then give the barriers something to overlap (cross-barrier weight prefetch), and last bring the tiles above 3 TB/s. The first two steps are compiler and runtime work, which is where ETX’s value lies; the third is tile engineering, which sets the distance to vLLM.
- **Budget:** 7 phases, about 10 GPU sessions of 1-2 hours on a 1x MI300X at \$2.99/h, \$40-60 in total. The balance is \$26.20 and needs topping up.

1. Where we are and why

Same MI300X, 1024-token context, batch 1 (2026-09-25):

Model	vLLM best	ETX unfused (same tiles, HIP graph)	ETX fused	Hand-written fleet
Qwen2.5-1.5B	1.86 ms	4.05	4.47	-
Qwen3-8B	4.91 ms	9.45	10.07	-
Qwen3-30B-A3B	4.84 ms	9.07	10.48	-
DeepSeek-V2-Lite	4.12 ms	not measured	3.78 (fleet’s tiles)	3.57

Per-phase trace of Qwen3-8B: 274 us per layer against a bandwidth floor of about 98 us.

Where it goes	Time	Root cause
The four GEMV phases	190 us	1.5-2.5 TB/s effective: small tasks, too few loads in flight
Attention plus merge	63 us	Serial dot products per thread after chunking by position; merge reads 64 chunks serially
Eight device-wide barriers	about 9 us more than kernel boundaries	300 workers poll one counter, the last arriver writes back L2, eight XCDs each invalidate

Three root causes: 1. **The graph has the wrong shape.** The Qwen graph has eight device-wide barriers per layer. Fleet’s has two; the other six are XCD-local events. The Qwen graph does not use ETX’s most valuable ability, building the graph around the chiplets. 2. **A barrier costs more than a kernel boundary,** about 2 us more each. Batch-1 decode is a chain in which every link needs the whole output of the previous one, so the number of barriers is fixed by the model; each one must get cheaper, or the waiting time must be put to use. 3. **The tiles are weak.** GEMV and attention are latency-bound. That has nothing to do with fusion, but it sets the distance to vLLM.

2. Design: one slice of every layer per XCD

Fleet's hand-made scheme generalises into a compiler rule. A layer's work is cut into eight slices, one per XCD; inside a slice only local events are used, and the slices meet at two points only:

```
        inside XCD k (local events, L1 invalidate only)
x --norm--> slice k of the qkv rows --> attention for this XCD's KV-head group --> merge --
> K-slice k of o_proj                                                                    | writes
this XCD's partial sum o_k
        ===== device-wide barrier 1: eight partial sums ready =====
x + sum(o_k) --norm--> slice k of the gate_up rows --> K-slice k of down --> writes this
XCD's partial sum d_k
        ===== device-wide barrier 2: eight partial sums ready =====
prologue of the next layer's qkv: x = x + sum(d_k)    (reads eight vectors: cheaper than a
barrier)
```

Key points: - **Rows go to XCDs, K goes to XCDs.** qkv and gate_up are cut by output rows; each slice is exactly what the same XCD needs next. o_proj and down are cut by K; each XCD consumes only the input its own slice produced and writes a partial sum. - **Summing the partials happens in the next phase's prologue.** This is fleet's FOLD_PARTIALS and the design document's "recompute for events": reading 8 x H floats is far cheaper than a device-wide barrier. - **Attention follows the KV-head groups.** Under GQA each XCD holds NKV/8 KV heads and their query heads, so KV-cache reads and writes stay inside the XCD. - **Weights are packed per slice.** The packer lays out each XCD's row slice and K slice as one contiguous block, so a task streams contiguous memory.

Device-wide barriers per layer drop from eight to two. Everything else is an XCD-local event at about 1 us with no L2 invalidate.

3. Phases

Phase 0: complete the measurements (1 session)

- Measure "fleet's tiles, unfused" on DeepSeek. That number decides whether a locality-aware graph makes fusion pay.
- Barrier microbenchmark at 304 workgroups: the ETX device-wide barrier, a two-level barrier, and a HIP-graph kernel boundary, in microseconds. The numbers go into the cost table of gfx942.yaml.
- Done when: the three numbers are in the table and the DeepSeek unfused figure is in the comparison table.

Phase 1: the compiler builds XCD-local graphs (2 sessions)

- **P2 extension, "slice by domain":** when a grid's output is consumed per domain downstream, place the producer slices per domain and lower the event to DOMAIN scope. The input is an edge map, for example qkv row blocks to KV-head groups as "i->(i/rows_per_group)".
- **Sliced graph builder:** examples/llm/model.py emits the grids and events of section 2; partial-sum buffers of shape [8][H]; the fold as a Pass-3 recompute prologue.
- **Per-slice weight packing:** prep.py writes slice-contiguous layouts and the manifest records slice offsets.
- Done when: all three Qwen models still match HF token for token; two device-wide barriers per layer; the fused kernel at least equals the unfused one.

Phase 2: a two-level device-wide barrier (1 session)

- Arrival in two levels: first the XCD's counter, then the XCD's last arriver performs one L2 write-back and arrives at the device counter. This half exists already.
- Waiting in two levels: one waiter per XCD polls the device counter; the other workers poll a word local to their XCD. When the waiter sees completion it invalidates its XCD's L2 first, then writes the local word; the others only invalidate L1. This differs from the relay shown unsafe: the invalidate is done by a worker inside the XCD, the waiter waits for it to complete with `s_waitcnt` before writing the local word, and every other worker still performs its own L1 invalidate after reading the word. The scheme is enabled only after it passes the Qwen2.5 determinism test.
- Done when: the barrier costs about 1 us; Qwen2.5 produces identical tokens over three runs.

Phase 3: cross-barrier weight prefetch (2 sessions)

This is what a sequence of independent kernels structurally cannot do, and where fusion actually pulls ahead.

- A worker's next task is known from its static queue, so the weight addresses are known before the wait. The wait loop interleaves loads of the next task's weight block, into L2 (touch loads) or straight into an LDS staging area.
- LDS budget: dense models run one workgroup per CU; of the 64 KB the input vector takes 16-48 KB and the rest stages weights; when that is too small, fall back to warming L2.
- This is Pass 7 of the design; the machine model marks `async_copy_to_lds` as none on gfx942, so the degraded implementation is ordinary loads into LDS.
- Done when: the GEMV start-up after a barrier (first-ready to body start in the trace) shrinks visibly; fused beats unfused by at least 15%.

Phase 4: merge the small phases (1 session)

- post (q/k norm, RoPE, KV write) folds into qkv's epilogue: each qkv task covers a whole number of heads and finishes them in place.
- merge is done by the last attention task of each head to finish (the last-arriver pattern the runtime already supports).
- Done when: phases per layer drop from 8 to 5 and their 30-40 us disappear.

Phase 5: tiles above 3 TB/s (2-3 sessions)

- **GEMV:** keep the input vector in LDS as bf16 to halve its footprint so dense models also fit two workgroups per CU; weight blocks of 256 KB or more per task; 32 loads in flight; non-temporal loads on by default. Target 3.2 TB/s, against vLLM's 3.0 TB/s on the 8B model.
- **Attention:** a flash-decode layout, four waves over position ranges with vectorised 128-dimension dot products, partial sums out; target under 5 us per layer at 1024 context.
- **lm_head:** already at 3 TB/s, unchanged.
- The alternative is linking vLLM or AITER GEMV and attention as tiles. ETX's link mode accepts any HIP function, but those are whole kernels and would have to be cut into tiles; kept as a fallback.
- Done when: fused Qwen3-8B under 6 ms.

Phase 6: comparison and delivery (1 session)

- Four models x {vLLM best, unfused, fused}, 1024 context, three rounds each, every run checked token by token against HF.

- Update the design document, the brief and the README; put the barrier costs and prefetch gains into the machine model's cost table.

4. Expected gains (Qwen3-8B, per token)

Phase	Fused, expected	Basis
Today	10.07 ms	measured
Phase 1, barriers 8 to 2	about 9.3 ms	six barriers plus waits of about 5 us saved per layer, 36 layers
Phase 2, two-level barrier	about 9.2 ms	1 us saved on each remaining barrier
Phase 3, prefetch	about 8.3 ms	about 10 us of GEMV start-up hidden after every barrier
Phase 4, merged phases	about 7.2 ms	30 us saved per layer
Phase 5, tiles at 3.2 TB/s	about 5.5 ms	15.1 GB / 3.2 TB/s plus the remaining synchronisation

Unfused is expected at 6.3 ms after phase 5; vLLM is at 4.91 ms. So: beating unfused by 15% rests on phases 1-4; catching vLLM rests on phase 5; beating vLLM by 15% needs tiles above 3.6 TB/s, or the larger routing gain available on MoE. The MoE model should do better: with 128 experts and top-8, the expert weights can be prefetched the moment the routing is known, which independent kernels cannot do.

5. Risks

- **Correctness of the two-level barrier.** The relay's hierarchical acquire was shown unsafe. The new scheme must pass the Qwen2.5 determinism test; if it does not, only the arrival side stays two-level and waiting stays per worker.
- **Prefetch competes for LDS and registers.** It can lower the GEMV's loads in flight; the resource accounting of Pass 1 must allocate both, with the depth decided by A/B.
- **The sliced graph assumes divisibility.** Head counts or expert counts not divisible by eight need uneven slices; the builder must handle remainders.
- **Tile speed-up is the largest uncertainty.** 3.2 TB/s is an estimate; if it is missed, ETX still trails vLLM, but goal 1 is unaffected.
- **GPU availability.** Hot Aisle has often had no 1x or 2x VM for hours at a time.

6. Order and the decision point

Phases 0 and 1 come first. Phase 0 takes 20 minutes and answers whether a locality-aware graph makes fusion pay; phase 1 is the foundation of everything after it and the most valuable single rule in the compiler. If, after those two, the fused kernel still cannot equal the unfused one, the right move is to reconsider whether batch-1 decode is the correct target for a megakernel at all and to shift the weight to MoE routing and multi-GPU, where independent kernels cannot follow.